A4.4 Memory types

The AXI4 protocol introduces new names for the *memory types* identified by the **AxCACHE** encoding. Table A4-5 shows the AXI4 **AxCACHE** encoding and associated memory types. Some memory types have different encodings in AXI3 and these are shown in brackets.

Note

The same memory type can have different encodings on the read channel and write channel. This provides backwards compatibility with AXI3 **AxCACHE** definitions.

In AXI4 it is legal to use more than one **AxCACHE** value for a particular memory type. Table A4-5 shows the preferred AXI4 value with the legal AXI3 value in brackets.

Table A4-5 Memory type encoding

ARCACHE[3:0]	AWCACHE[3:0]	Memory type
0000	0000	Device Non-bufferable
0001	0001	Device Bufferable
0010	0010	Normal Non-cacheable Non-bufferable
0011	0011	Normal Non-cacheable Bufferable
1010	0110	Write-through No-allocate
1110 (0110)	0110	Write-through Read-allocate
1010	1110 (1010)	Write-through Write-allocate
1110	1110	Write-through Read and Write-allocate
1011	0111	Write-back No-allocate
1111 (0111)	0111	Write-back Read-allocate
1011	1111 (1011)	Write-back Write-allocate
1111	1111	Write-back Read and Write-allocate

All values not shown in Table A4-5 are reserved.

A4.4.1 Memory type requirements

This section specifies the required behavior for each of the memory types.

Device Non-bufferable

The required behavior for Device Non-bufferable memory is:

- The write response must be obtained from the final destination.
- Read data must be obtained from the final destination.
- Transactions are Non-modifiable, see *Non-modifiable transactions* on page A4-60
- Reads must not be prefetched. Writes must not be merged.
- All Non-modifiable read and write transactions (**AxCACHE[1]** = 0) from the same ID to the same slave must remain ordered.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

Device Bufferable

The required behavior for the Device Bufferable memory type is:

- The write response can be obtained from an intermediate point.
- Write transactions must be made visible at the final destination in a timely manner, as defined in *Transaction buffering* on page A4-70.
- Read data must be obtained from the final destination.
- Transactions are Non-modifiable, see *Non-modifiable transactions* on page A4-60.
- Reads must not be prefetched. Writes must not be merged.
- All Non-modifiable read and write transactions (**AxCACHE[1]** = 0) from the same ID to the same slave must remain ordered.

Note

Both Device memory types are Non-modifiable. In this protocol specification the terms Device memory and Non-modifiable memory are interchangeable.

For read transactions there is no difference in the required behavior for Device Non-bufferable and Device Bufferable memory types.

Normal Non-cacheable Non-bufferable

The required behavior for the Normal Non-cacheable Non-bufferable memory type is:

- the write response must be obtained from the final destination
- read data must be obtained from the final destination
- transactions are Modifiable, see *Modifiable transactions* on page A4-61
- writes can be merged
- read and write transactions from the same ID to addresses that overlap must remain ordered.

Normal Non-cacheable Bufferable

The required behavior for the Normal Non-cacheable Bufferable memory type is:

- The write response can be obtained from an intermediate point.
- Write transactions must be made visible at the final destination in a timely manner, as defined in *Transaction buffering* on page A4-70. There is no mechanism to determine when a write transaction is visible at its final destination.
- Read data must be obtained either:
  - from the final destination
  - from a write transaction that is progressing to its final destination.

  If read data is obtained from a write transaction:
  - it must be obtained from the most recent version of the write
  - the data must not be cached to service a later read.
- Transactions are Modifiable, see *Modifiable transactions* on page A4-61.
- Writes can be merged.
- Read and write transactions from the same ID to addresses that overlap must remain ordered.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

Note

For a Normal Non-cacheable Bufferable read, data can be obtained from a write transaction that is still progressing to its final destination. This is indistinguishable from the read and write transactions propagating to arrive at the final destination at the same time. Read data returned in this manner does not indicate that the write transaction is visible at the final destination.

Write-through No-allocate

The required behavior for the Write-through No-allocate memory type is:

- The write response can be obtained from an intermediate point.
- Write transactions must be made visible at the final destination in a timely manner, as defined in *Transaction buffering* on page A4-70. There is no mechanism to determine when a write transaction is visible at the final destination.
- Read data can be obtained from an intermediate cached copy.
- Transactions are Modifiable, see *Modifiable transactions* on page A4-61.
- Reads can be prefetched.
- Writes can be merged.
- A cache lookup is required for read and write transactions.
- Read and write transactions from the same ID to addresses that overlap must remain ordered.
- The No-allocate attribute is an allocation hint, that is, it is a recommendation to the memory system that, for performance reasons, these transactions are not allocated. However, the allocation of read and write transactions is not prohibited.

Write-through Read-allocate

The required behavior for the Write-through Read-allocate memory type is the same as for Write-through No-allocate memory. But in this case the allocation hint is that, for performance reasons:

- allocation of read transactions is recommended
- allocation of write transactions is not recommended.

Write-through Write-allocate

The required behavior for the Write-through Write-allocate memory type is the same as for Write-through No-allocate memory. But in this case the allocation hint is that, for performance reasons:

- allocation of read transactions is not recommended
- allocation of write transactions is recommended.

Write-through Read and Write-allocate

The required behavior for the Write-through Read and Write-allocate memory type is the same as for Write-through No-allocate memory. But in this case the allocation hint is that, for performance reasons:

- allocation of read transactions is recommended
- allocation of write transactions is recommended.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

Write-back No-allocate

The required behavior for the Write-back No-allocate memory type is:

- The write response can be obtained from an intermediate point.
- Write transactions are not required to be made visible at the final destination.
- Read data can be obtained from an intermediate cached copy.
- Transactions are Modifiable, see *Modifiable transactions* on page A4-61.
- Reads can be prefetched.
- Writes can be merged.
- A cache lookup is required for read and write transactions.
- Read and write transactions from the same ID to addresses that overlap must remain ordered.
- The No-allocate attribute is an allocation hint, that is, it is a recommendation to the memory system that, for performance reasons, these transactions are not allocated. However, the allocation of read and write transactions is not prohibited.

Write-back Read-allocate

The required behavior for the Write-back Read-allocate memory type is the same as for Write-back No-allocate memory. But in this case the allocation hint is that, for performance reasons:

- allocation of read transactions is recommended
- allocation of write transactions is not recommended.

Write-back Write-allocate

The required behavior for the Write-back Write-allocate memory type is the same as for Write-back No-allocate memory. But in this case the allocation hint is that, for performance reasons:

- allocation of read transactions is not recommended
- allocation of write transactions is recommended.

Write-back Read and Write-allocate

The required behavior for the Write-back Read and Write-allocate memory type is the same as for Write-back No-allocate memory. But in this case the allocation hint is that, for performance reasons:

- allocation of read transactions is recommended
- allocation of write transactions is recommended.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential